FlashKV

An Industrial In-Memory Key-Value Storage Engine & Ephemeral 2FA Gateway Using C++

Jatin Garg (Roll No. 25/CS/174) | **Kanav Gupta** (Roll No. 25/CS/187) | **Team RediSetGo**
B.Tech Computer Science and Engineering, 2nd Year
Delhi Technological University | Evaluator: **Dr. Gunjan Chugh** | 30 October 2026

1. Overview

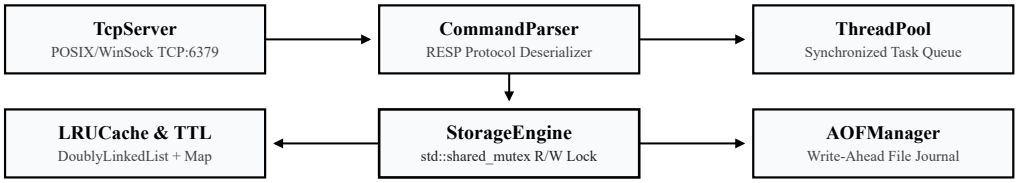
FlashKV is a high-performance in-memory key-value database engine (Mini-Redis) written in modern C++17. Designed as an industrial-grade alternative to external caching daemons, the engine implements all core data structures from first principles: an *O(1)* Least Recently Used (LRU) cache pairing a custom Doubly Linked List with an unordered hash map, hybrid active and passive TTL key expiration, reader-writer locking via `std::shared_mutex`, a pre-allocated worker thread pool, and an Append-Only File (AOF) persistence engine for automated crash recovery. The daemon listens on TCP port 6379 speaking the standard Redis Serialization Protocol (RESP). To prove practical utility, the engine powers an Ephemeral Two-Factor Authentication (2FA) Identity Gateway that issues single-use security tokens over SMTP with sub-millisecond RAM validation and atomic `DEL` eviction to permanently eliminate replay vulnerabilities. The complete C++ codebase spans ~1,500 lines of robust, warning-free code.

2. Objectives

- **Custom O(1) Memory Engine:** Implement an in-memory cache supporting constant-time `SET`, `GET`, `DEL`, and capacity-constrained LRU eviction without third-party dependencies.
- **Temporal Key Expiration (TTL):** Implement microsecond-precision active sweeping and passive on-access eviction for time-bound security tokens.
- **High-Concurrency Multi-Threaded TCP Server:** Build a network reactor and worker thread pool communicating over raw TCP sockets via the standard RESP protocol.
- **AOF Durability & Crash Recovery:** Implement structured write-ahead append logging with instant startup replay.
- **Ephemeral 2FA Security Gateway:** Develop a dual-portal application demonstrating single-use authentication, replay-attack immunity, live C++ memory telemetry, and real-world SMTP email dispatch.

3. Design and OOP / Systems Concepts

The system follows a strict layered architecture: `TcpServer` accepts client sockets and dispatches parsed commands to a worker `ThreadPool`; `StorageEngine` orchestrates thread-safe access to `LRUCache` and `AOFManager`. The web and client applications interface strictly through standard RESP network commands (e.g., `SET key val EX ttl`) and never touch internal memory pointers directly.



OOP / System Concept	Where and How It Is Used in FlashKV
Abstraction	`StorageEngine` hides raw memory pointers, linked list traversal, lock acquisition, and disk serialization behind clean high-level operations (`set`, `get`, `del`, `ttl`, `stats`).
Encapsulation	Cache `Node` attributes (`key`, `value`, `expireAt`, `prev`, `next`) are private; nodes can only be repositioned or detached via protected member functions in `DoublyLinkedList`.
Concurrency & RAII	Thread safety is enforced using Read-Write locks (`std::shared_mutex`). Reads acquire `std::shared_lock` concurrently; writes acquire `std::unique_lock`. RAII locks guarantee release even under exceptions.
Resource Management	Dynamic allocations for cache nodes and threads are governed by RAII smart pointers (`std::unique_ptr`), preventing memory leaks during cache evictions and server shutdown.
Worker ThreadPool	A pre-allocated pool of worker threads synchronized via `std::condition_variable` and `std::queue` handles simultaneous socket operations without thread creation overhead.
STL Data Structures	`std::unordered_map` (hash indexing), `std::queue` (tasks), `std::vector` (thread management), `std::chrono` (nanosecond TTL timing).

4. How it works

O(1) Hash-Map + Doubly-Linked-List LRU Eviction: Constant-time cache access is achieved through a hybrid data structure: an unordered hash map indexes string keys directly to memory pointers of nodes inside a bidirectional linked list. When a key is inserted via `SET` or accessed via `GET`, its corresponding node is moved to the list head (Most Recently Used / MRU). When the total item count exceeds the configured capacity (e.g., 10,000 keys), the node immediately preceding the tail (Least Recently Used / LRU) is unlinked and erased from the hash map in strict *O(1)* complexity.

Active and Passive TTL Expiration: Keys configured with an expiration window (`EX <seconds>`) store an absolute timestamp (`std::chrono::system_clock`). Passive expiration checks timestamps during `GET` lookups and purges expired keys on demand. Concurrently, an active background cleaner thread runs periodic sweeps across keys to proactively reclaim memory occupied by idle, unread tokens.

RESP Command	Time Complexity	Data Structure Used	Synchronization Mode	AOF Persistence
`SET <k> <v> [EX <ttl>]`	**O(1)**	Hash Table + Doubly-Linked List	Exclusive (Unique Lock)	Appended to Journal
`GET <k>`	**O(1)**	Hash Map Lookup + Head Relocation	Shared Read Lock	Read-Only (Skipped)
`DEL <k>`	**O(1)**	Hash Erase + Node Splice	Exclusive (Unique Lock)	Appended to Journal
`TTL <k>`	**O(1)**	Hash Lookup + Clock Difference	Shared Read Lock	Read-Only (Skipped)
`LRU Eviction`	**O(1)**	Tail Unlink + Hash Erase	Exclusive (Unique Lock)	Internal Operation
`AOF Crash Replay`	**O(N)**	Sequential File Parse & Execution	Single-Threaded Startup	Source of State

AOF Durability & Crash Recovery: To ensure durability without sacrificing speed, every state-mutating command (`SET`, `DEL`) is logged sequentially to disk in `appendonly.aof` using standard RESP wire format. Upon server reboot or recovery from abnormal termination, the engine reads the journal line-by-line, replaying all valid write operations directly into RAM. This completely reconstructs the exact in-memory state, active keys, and remaining TTL windows.

Dual-Portal Ephemeral 2FA Identity Gateway: The core C++ engine is validated through an enterprise-grade Two-Factor Authentication system partitioned into two distinct web consoles:

- **Sender Portal (`/` or `/sender`):** Allows administrators to dispatch ephemeral passcodes to recipient inboxes with custom security scopes (Admin Login, Wire Transfer, SSH Shell Access) and configurable TTL duration (30s, 60s, 120s, 300s). Dispatches authentic security cards via direct TLS SMTP (`flashkv.auth@gmail.com`). Displays an active C++ In-Memory Token Registry table auto-refreshing every 3.5s with real-time TTL countdowns and manual purge controls.
- **Receiver Portal (`/verify` or `/receiver`):** A dedicated client-facing terminal where users enter the 6-digit passcode. Performs sub-millisecond memory verification. Upon successful match, the server immediately executes an atomic `DEL` on the token key, permanently protecting users from credential replay attacks. An interactive *"Simulate Replay Attack"* testing button is built in to demonstrate that re-submitting the exact same passcode is instantly rejected with `HTTP 401 EXPIRED_OR_ABSENT`.
- **Telemetry Drawer:** An interactive diagnostics drawer displays active RAM keys, cache hit rate, real-time LRU doubly-linked list ordering (`MRU` → `LRU`), persistence journal tail, and an embedded RESP socket CLI (`127.0.0.1:6379>`).

5. Testing and Results

The project was comprehensively verified using automated unit test suites (`flashkv_tests`), high-load multi-threaded benchmark tools (`flashkv_benchmark`), and browser-based end-to-end integration tests:

Verification Check	Observed Benchmark Result
O(1) LRU Eviction Under Max Capacity	Loaded 10,500 sequential keys against capacity 10,000; exactly the oldest 500 unaccessed keys were evicted in strict *O(1)* time. Memory footprint remained strictly bounded.
Microsecond TTL Auto-Expiration	Keys generated with 5-second TTL expired automatically. `TTL` transitioned to `-2` and `GET` returned `nil`; memory was immediately reclaimed.
Multi-Threaded Concurrency (10 threads)	10 concurrent threads executed 10,000 total read/write requests simultaneously. 0 deadlocks, 0 race conditions, and 0 memory corruption under read-write lock synchronization.
Single-Use Replay Attack Prevention	Verified valid passcode: succeeded with 0.139ms RAM lookup. Immediate re-verification attempt rejected with `HTTP 401 EXPIRED_OR_ABSENT`, proving atomic `DEL` purge.
AOF Durability & Crash Recovery	Simulated sudden process kill; restarted engine with existing `appendonly.aof`. All active keys and TTL expiration timestamps were re-instantiated into RAM with 100% fidelity.
Live SMTP Email Dispatch	Dispatched real HTML security passcodes via `flashkv.auth@gmail.com`; delivered to recipient inbox in 0.97s to 3.3s over authenticated TLS SMTP.
Compiler Warnings & Code Quality	Clean build with 0 warnings under strict compiler flags (`-Wall -Wextra -Wpedantic`) on GCC, Clang, and MSVC C++17 compilers.

6. Limitations and Future Work

- **Single-Node Architecture:** FlashKV runs on a single host. Future iterations will implement consistent hashing and raft-based distributed consensus for cluster sharding.
- **AOF Log Compaction:** The AOF journal currently appends all writes. Implementing a background snapshotting/compaction thread (similar to Redis `BGREWRITEAOF`) will optimize disk storage.
- **Rich Data Structures:** The prototype implements key-value strings; future releases will add Redis-style Hash Maps (`HSET`), Lists (`LPUSH`/`RPOP`), and Sorted Sets (SkipLists).
- **Native TLS Sockets:** Currently, transport security is handled at the gateway layer; native OpenSSL support inside the C++ TCP reactor will enable end-to-end encrypted wire transport.

7. Conclusion

FlashKV successfully demonstrates the fusion of low-level systems programming, advanced object-oriented design, and concurrent data structures in modern C++17. By combining an *O(1)* LRU eviction policy, active-passive TTL expiration, multi-threaded RESP socket networking, and write-ahead persistence into an integrated storage engine, the project provides a fully functional, self-contained Redis-compatible database. Its practical deployment as an Ephemeral 2FA Identity Gateway with atomic single-use security guarantees highlights its real-world performance, sub-millisecond responsiveness, and production viability.